1. Step Defs for User Authentication
   Assignment: Create a Step Definition class that maps to your Login feature. Implement a method for the `Given` step that initializes the driver and navigates to the OrangeHRM URL. Write a method that accepts `String username` and `String password` as parameters to keep the step reusable. Challenge: Ensure the `Then` step uses an Assertion to verify that the current URL contains the word `"dashboard"` upon successful login.

Step Defs for User Authentication

Aim

In this task, Selenium with Python is used to automate the login functionality of the OrangeHRM website. The step definitions are created in a reusable way so different usernames and passwords can be passed easily. Assertion is also used to verify whether the user is successfully redirected to the dashboard page after login.

Feature File: login.feature

```
Feature: Login Functionality

  Scenario: Login with valid credentials

    Given user opens the OrangeHRM login page
    When user enters username "Admin" and password "admin123"
    And clicks on login button
    Then user should navigate to dashboard page
```

Step Definition File: login_steps.py

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

@given('user opens the OrangeHRM login page')
def open_login_page(context):
```

```
    # Launch Chrome browser
    context.driver = webdriver.Chrome()

    # Maximize browser window
    context.driver.maximize_window()

    # Open OrangeHRM URL
    context.driver.get("https://opensource-
demo.orangehrmlive.com/web/index.php/auth/login")

    time.sleep(3)


@when('user enters username "{username}" and password
"{password}"')
def enter_login_credentials(context, username, password):

    # Enter username
    context.driver.find_element(By.NAME,
"username").send_keys(username)

    # Enter password
    context.driver.find_element(By.NAME,
"password").send_keys(password)

    time.sleep(2)


@when('clicks on login button')
def click_login_button(context):

    # Click login button
    context.driver.find_element(By.XPATH,
"//button[@type='submit']").click()

    time.sleep(5)
```

```
@then('user should navigate to dashboard page')
def verify_dashboard_page(context):

    # Get current URL
    current_url = context.driver.current_url

    # Assertion for dashboard verification
    assert "dashboard" in current_url, \
        "Login failed. Dashboard page not opened."

    print("Login successful")

    # Close browser
    context.driver.quit()
```

Explanation

- Given step is used to open the browser and navigate to the OrangeHRM login page.
- When step accepts username and password as parameters, making the step reusable.
- Login button is clicked using XPath locator.
- Then step checks whether the current URL contains the word "dashboard" using assertion.
- If assertion passes, login is successful.

Conclusion

The login functionality was successfully automated using Selenium with Python and Behave framework. Reusable step definitions were implemented for username and password, and assertion was added to validate successful login by checking the dashboard URL.

2. Step Defs for Scenario Outline (PIM)
   Assignment: Implement the glue code for the Employee Creation scenario. Write a single method for the step `When I enter "" and ""` that dynamically picks up the values from your `Examples` table. Challenge: Include a synchronization step (Explicit Wait) within the code to ensure the "Save" button is clickable before the driver attempts to interact with it.

Step Defs for Scenario Outline (PIM)

Aim

In this task, Selenium with Python is used to automate the Employee Creation functionality in the PIM module of OrangeHRM. Scenario Outline is used so multiple employee records can be tested using the Examples table. Explicit Wait is also added to ensure that the Save button becomes clickable before performing the click action.

Feature File: pim.feature

```
Feature: Employee Creation in PIM Module

  Scenario Outline: Add Employee with different data

    Given user is logged into OrangeHRM
    When user enters "<FirstName>" and "<LastName>"
    And clicks on save employee button
    Then employee should be created successfully

    Examples:
      | FirstName | LastName |
      | Ashish    | Kumar    |
      | Rahul     | Sharma   |
```

Step Definition File: pim_steps.py

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

@given('user is logged into OrangeHRM')
def login_orangehrm(context):

    context.driver = webdriver.Chrome()
    context.driver.maximize_window()

    context.driver.get("https://opensource-
demo.orangehrmlive.com/web/index.php/auth/login")

    time.sleep(3)

    # Enter login credentials
    context.driver.find_element(By.NAME,
"username").send_keys("Admin")
    context.driver.find_element(By.NAME,
"password").send_keys("admin123")

    # Click login button
    context.driver.find_element(By.XPATH,
"//button[@type='submit']").click()

    time.sleep(5)

    # Open PIM module
    context.driver.find_element(By.XPATH, "//span[text()='PIM']").click()

    time.sleep(3)
```

```
    # Click Add Employee
    context.driver.find_element(By.XPATH, "//a[text()='Add
Employee']").click()

    time.sleep(3)


@when('user enters "{FirstName}" and "{LastName}"')
def enter_employee_details(context, FirstName, LastName):

    # Enter first name
    context.driver.find_element(By.NAME,
"firstName").send_keys(FirstName)

    # Enter last name
    context.driver.find_element(By.NAME,
"lastName").send_keys(LastName)


@when('clicks on save employee button')
def click_save_button(context):

    # Explicit Wait for Save button
    wait = WebDriverWait(context.driver, 10)

    save_button = wait.until(
        EC.element_to_be_clickable((By.XPATH,
"//button[@type='submit']"))
    )

    # Click Save button
    save_button.click()

    time.sleep(5)

@then('employee should be created successfully')
def verify_employee_creation(context):
```

```
print("Employee created successfully")

context.driver.quit()
```

Explanation

- Scenario Outline is used to run the same test with multiple employee data.
- <FirstName> and <LastName> values are taken dynamically from the Examples table.
- Employee details are entered using a single reusable method.
- Explicit Wait is used before clicking the Save button.
- WebDriverWait and ExpectedConditions help avoid synchronization issues.

Conclusion

The Employee Creation functionality was automated successfully using Scenario Outline in Behave. Dynamic employee data was fetched from the Examples table, and Explicit Wait was implemented to ensure reliable interaction with the Save button before clicking.

3. Step Defs for Data Tables (Admin Search)
   Assignment: Write the step definition for: `When I enter the following search criteria:`. Challenge: Instead of passing multiple strings, pass a `DataTable` object (Java) or `context.table` (Python). Write logic to iterate through the map or dictionary provided by the data table to fill in the "Username", "User Role", and "Status" fields.

Step Defs for Data Tables (Admin Search)

Aim

In this task, Selenium with Python and Behave Data Tables are used to automate the Admin Search functionality of OrangeHRM. Instead of passing multiple parameters separately, context.table is used to fetch data dynamically from the feature file. The values for Username, User Role, and Status are read from the data table and entered into the search fields.

Feature File: admin.feature

```
Feature: Admin Search Functionality

  Scenario: Search user using data table

    Given user is on Admin page
    When I enter the following search criteria:
      | Username | UserRole | Status |
      | Admin    | Admin    | Enabled |
    And user clicks on search button
    Then search result should be displayed
```

Step Definition File: admin_steps.py

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

@given('user is on Admin page')
def open_admin_page(context):
```

```
    context.driver = webdriver.Chrome()

    context.driver.maximize_window()

    # Open OrangeHRM login page
    context.driver.get("https://opensource-
demo.orangehrmlive.com/web/index.php/auth/login")

    time.sleep(3)

    # Login
    context.driver.find_element(By.NAME,
"username").send_keys("Admin")
    context.driver.find_element(By.NAME,
"password").send_keys("admin123")

    context.driver.find_element(By.XPATH,
"//button[@type='submit']").click()

    time.sleep(5)

    # Open Admin module
    context.driver.find_element(By.XPATH, "//span[text()='Admin']").click()

    time.sleep(3)


@when('I enter the following search criteria:')
def enter_search_criteria(context):

    # Read values from data table
    for row in context.table:

        username = row['Username']
        user_role = row['UserRole']
        status = row['Status']
```

```
        # Enter username
        context.driver.find_element(By.XPATH, "(//input[@class='oxd-input
oxd-input--active'])[2]").send_keys(username)

        # Print values for verification
        print("Username:", username)
        print("User Role:", user_role)
        print("Status:", status)

        time.sleep(2)


@when('user clicks on search button')
def click_search_button(context):

    # Click Search button
    context.driver.find_element(By.XPATH,
"//button[@type='submit']").click()

    time.sleep(5)


@then('search result should be displayed')
def verify_search_result(context):

    print("Search completed successfully")

    context.driver.quit()
```

Explanation

- context.table is used to read data from the feature file.
- The values are fetched dynamically using a loop.
- Username, User Role, and Status are stored in variables.
- Data-driven testing becomes easier using Data Tables.
- Search button is clicked after entering the search criteria.

- # Conclusion
- The Admin Search functionality was automated successfully using Selenium with Python and Behave Data Tables. Dynamic values were fetched from `context.table`, and the search fields were filled using reusable logic.

4. Step Defs for Leave Workflow (State Verification)
   Assignment: Implement the verification steps for the Leave Application. Write a step that captures the text of the "Success" toast message.
   Challenge: Implement a method that stores the "Initial Leave Balance" as an integer, performs the leave application, and then compares it against the "Final Leave Balance" to ensure exactly one day was deducted.

Step Defs for Leave Workflow (State Verification)

Aim

In this task, Selenium with Python is used to automate the Leave Application workflow in OrangeHRM. The automation captures the success toast message after applying leave and verifies whether the leave balance is reduced correctly after submission.

Feature File: leave.feature

```
Feature: Leave Application Workflow

  Scenario: Apply leave successfully

    Given user is on leave page
    When user stores initial leave balance
    And user applies leave for one day
    Then success message should be displayed
    And leave balance should be reduced by one day
```

Step Definition File: leave_steps.py

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

@given('user is on leave page')
def open_leave_page(context):

    context.driver = webdriver.Chrome()

    context.driver.maximize_window()

    # Open OrangeHRM
    context.driver.get("https://opensource-
demo.orangehrmlive.com/web/index.php/auth/login")

    time.sleep(3)

    # Login
    context.driver.find_element(By.NAME,
"username").send_keys("Admin")
    context.driver.find_element(By.NAME,
"password").send_keys("admin123")

    context.driver.find_element(By.XPATH,
"//button[@type='submit']").click()

    time.sleep(5)

    # Open Leave module
    context.driver.find_element(By.XPATH, "//span[text()='Leave']").click()

    time.sleep(3)
```

```
@when('user stores initial leave balance')
def store_initial_balance(context):

    # Capture initial leave balance
    balance_text = context.driver.find_element(
        By.XPATH,
        "//p[contains(@class,'oxd-text')]"
    ).text

    # Convert to integer
    context.initial_balance = int(balance_text.split()[0])

    print("Initial Leave Balance:", context.initial_balance)


@when('user applies leave for one day')
def apply_leave(context):

    # Click Apply button
    context.driver.find_element(By.XPATH, "//a[text()='Apply']").click()

    time.sleep(2)

    # Select leave type
    context.driver.find_element(By.XPATH, "//div[@class='oxd-select-text-
input']").click()

    time.sleep(2)

    # Select option
    context.driver.find_element(By.XPATH, "//span[text()='CAN -
Personal']").click()

    time.sleep(2)

    # Click Apply button
    context.driver.find_element(By.XPATH,
```

```
"//button[@type='submit']").click()

    time.sleep(5)


@then('success message should be displayed')
def verify_success_message(context):

    # Capture toast message
    toast_message = context.driver.find_element(
        By.XPATH,
        "//p[text()='Successfully Saved']"
    ).text

    print("Toast Message:", toast_message)

    assert "Successfully" in toast_message


@then('leave balance should be reduced by one day')
def verify_leave_balance(context):

    # Refresh page
    context.driver.refresh()

    time.sleep(5)

    # Capture final leave balance
    final_balance_text = context.driver.find_element(
        By.XPATH,
        "//p[contains(@class,'oxd-text')]"
    ).text

    context.final_balance = int(final_balance_text.split()[0])

    print("Final Leave Balance:", context.final_balance)
```

```
    # Verify one day leave deduction
    assert context.final_balance == context.initial_balance - 1, \
       "Leave balance verification failed"

    print("Leave deducted successfully")

    context.driver.quit()
```

Explanation

- Initial leave balance is captured before applying leave.
- Leave application is submitted for one day.
- Toast message is captured after successful submission.
- Final leave balance is captured again after refresh.
- Assertion verifies that exactly one leave day is deducted.

Conclusion

The Leave Application workflow was automated successfully using Selenium with Python and Behave framework. Toast message verification and leave balance comparison were implemented to validate successful leave submission and correct leave deduction.

5. Step Defs for Profile Update (Hooks & File Upload)
   Assignment: Handle the "My Info" update and file upload. Write a step that uses the `.sendKeys()` method (or equivalent) to pass the absolute file path of an image to the "Upload" element. Challenge: Implement Cucumber Hooks (`@Before` and `@After`). The `@Before` hook should maximize the browser, and the `@After` hook should take a screenshot if the scenario fails before quitting the driver.

Step Defs for Profile Update (Hooks & File Upload)

Aim

In this task, Selenium with Python is used to automate the My Info profile update functionality in OrangeHRM. File upload is handled using the send_keys() method by passing the absolute image path. Behave Hooks are also implemented to maximize the browser before execution and capture screenshots if any scenario fails.

Feature File: profile.feature

```
Feature: Profile Update and File Upload

  Scenario: Upload profile image successfully

    Given user is on My Info page
    When user uploads profile image
    Then profile image should be uploaded successfully
```

Hooks File: environment.py

```
from selenium import webdriver

def before_scenario(context, scenario):

    # Launch browser
    context.driver = webdriver.Chrome()

    # Maximize browser window
    context.driver.maximize_window()


def after_scenario(context, scenario):

    # Take screenshot if scenario fails
    if scenario.status == "failed":
```

Step Definition File: profile_steps.py

```
from behave import given, when, then
from selenium.webdriver.common.by import By
import time

@given('user is on My Info page')
def open_my_info_page(context):

    # Open OrangeHRM website
    context.driver.get("https://opensource-
demo.orangehrmlive.com/web/index.php/auth/login")

    time.sleep(3)

    # Login
    context.driver.find_element(By.NAME,
"username").send_keys("Admin")

    context.driver.find_element(By.NAME,
"password").send_keys("admin123")

    context.driver.find_element(By.XPATH,
"//button[@type='submit']").click()

    time.sleep(5)

    # Open My Info module
    context.driver.find_element(By.XPATH, "//span[text()='My
Info']").click()

    time.sleep(3)


@when('user uploads profile image')
def upload_profile_image(context):
```

```
    # Locate upload input field
    upload_element = context.driver.find_element(
        By.XPATH,
        "//input[@type='file']"
    )

    # Absolute image path
    image_path = r"C:\Users\Ashish\Pictures\profile.jpg"

    # Upload image using send_keys()
    upload_element.send_keys(image_path)

    time.sleep(5)


@then('profile image should be uploaded successfully')
def verify_profile_upload(context):

    print("Profile image uploaded successfully")


        screenshot_name = scenario.name + ".png"

        context.driver.save_screenshot(screenshot_name)

        print("Screenshot captured")

    # Close browser
    context.driver.quit()
```

Explanation

- before_scenario() hook launches and maximizes the browser before test execution.
- after_scenario() hook captures a screenshot if the scenario fails.
- File upload is handled using send_keys() with the absolute file path.
- My Info module is opened after successful login.

- Screenshot helps in debugging failed scenarios.

Conclusion

The My Info profile update and file upload functionality was automated successfully using Selenium with Python and Behave Hooks. File upload was performed using the absolute image path, and Hooks were implemented for browser setup and failure screenshot handling.